

3.2. ARRAY DATA STRUCTURE

Properties

Insertion

Deletion

Searching: linear search, binary search

Merging

Sorting: selection sort, merge-sort, quick-sort

Merging (1)

- **Problem:** Given two sorted arrays, make a third sorted array containing a copy of all elements of the two original arrays.
- **Idea:** Compare the leftmost elements of the two arrays. Whichever element is less, copy it into the third array, and thereafter ignore that element.

Merging is one step in another sorting algorithm (mergesort) which we'll look at afterwards

Merging (2)

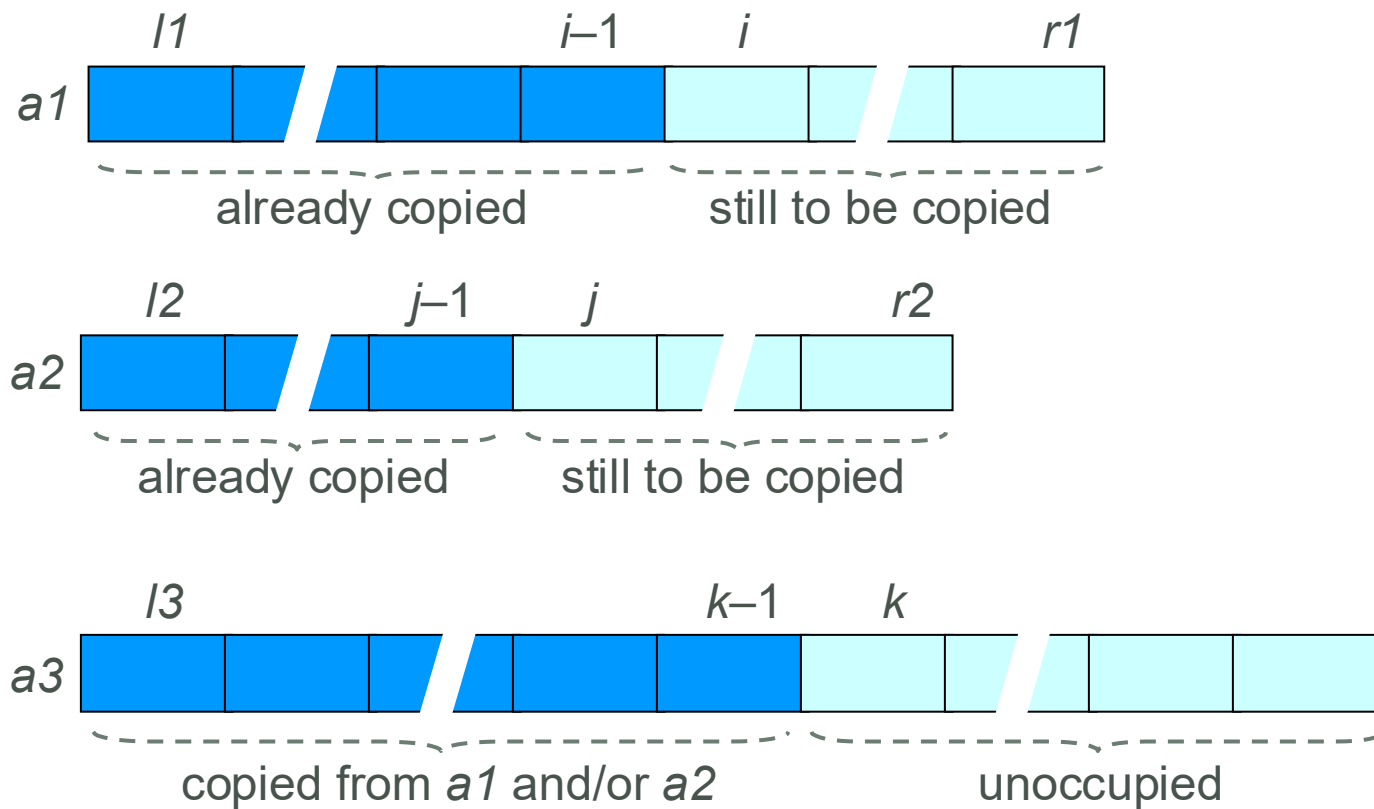
- **Array merging algorithm:**

To merge $a1[l1...r1]$ and $a2[l2...r2]$ into $a3[l3...r3]$
(where both $a1$ and $a2$ are sorted):

1. Set $i = l1$, set $j = l2$, and set $k = l3$.
2. While $i \leq r1$ and $j \leq r2$, repeat:
 - 2.1. If $a1[i]$ is less than or equal to $a2[j]$:
 - 2.1.1. Copy $a1[i]$ into $a3[k]$, then increment i and k .
 - 2.2. Else:
 - 2.2.1. Copy $a2[j]$ into $a3[k]$, then increment j and k .
3. If $i \leq r1$, copy $a1[i...r1]$ into $a3[k...r3]$.
4. If $j \leq r2$, copy $a2[j...r2]$ into $a3[k...r3]$.
5. Terminate.

Merging (3)

- Loop invariant:

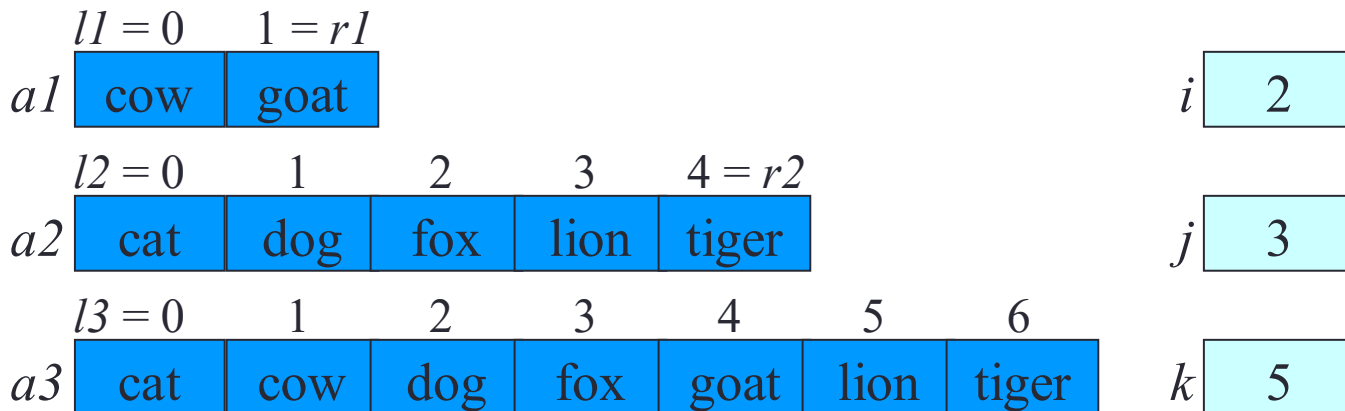


Merging (4)

- Animation:

To merge $a1[l1...r1]$ and $a2[l2...r2]$ into $a3[l3...]$:

1. Set $i = l1$, set $j = l2$, and set $k = l3$.
2. While $i \leq r1$ and $j \leq r2$, repeat:
 - ...
3. If $i \leq r1$, copy $a1[i...r1]$ into $a3[k...r3]$.
4. If $j \leq r2$, copy $a2[j...r2]$ into $a3[k...r3]$.
5. **Terminate.**



Merging (5)

- Analysis (**counting copies**):

Let n_1 and n_2 be the lengths of $a1$ and $a2$, $n = n_1 + n_2$.

Each element of $a1$ is copied once, and each element of $a2$ is copied once.

No. of copies = $n_1 + n_2 = n$

Time complexity is $O(n)$.

Merging (6)

- Analysis (**counting comparisons**):

Let n_1 and n_2 be the lengths of $a1$ and $a2$, $n = n_1 + n_2$.

Assume steps 2.1–2 perform a single comparison.

Steps 2.1–2 are repeated at most $n-1$ times.

Max. no. of comparisons = $n - 1$

Time complexity is again $O(n)$.

1. Set $i = l1$, set $j = l2$, and set $k = l3$.
2. While $i \leq r1$ and $j \leq r2$, repeat:
 - 2.1. If $a1[i]$ is less than or equal to $a2[j]$:
 - 2.1.1. Copy $a1[i]$ into $a3[k]$, then increment i and k .
 - 2.2. Else:
 - 2.2.1. Copy $a2[j]$ into $a3[k]$, then increment j and k .
3. If $i \leq r1$, copy $a1[i \dots r1]$ into $a3[k \dots r3]$.
4. If $j \leq r2$, copy $a2[j \dots r2]$ into $a3[k \dots r3]$.
5. Terminate.

Merging (7) – *Java implementation*

```
public static <E extends Comparable<E>> void merge (  
    E[] a1, int l1, int r1,  
    E[] a2, int l2, int r2,  
    E[] a3, int l3) {  
    // Merge the sorted subarrays a1[l1 - r1] and  
    // a2[l2 - r2] into a3[l3]  
    int i = l1, j = l2, k = l3;  
    while (i <= r1 && j <= r2) {  
        int comp = a1[i].compareTo(a2[j]);  
        if (comp <= 0) a3[k++] = a1[i++];  
        else a3[k++] = a2[j++];  
    }  
    while (i <= r1) a3[k++] = a1[i++];  
    while (j <= r2) a3[k++] = a2[j++];  
}
```



Explained on next slide

Merging (8)

Note that:

```
while (j <= r2)  a3[k++] = a2[j++];
```

is equivalent to

```
while (j <= r2)  {  
    a3[k] = a2[j]; k++; j++;  
}
```

Sorting

- **Problem:** Given an unsorted array, rearrange its elements into ascending order.
 - This is important because a sorted array can be searched and merged efficiently.
 - There is a huge choice of algorithms:
 - **selection sort**
 - **insertion sort**
 - **merge-sort**
 - **quick-sort**
 - shell-sort, radix sort, etc.
- 
- Diagram illustrating references for sorting algorithms:
- selection sort
 - insertion sort
 - merge-sort
 - quick-sort
 - shell-sort, radix sort, etc.
- see Watt & Brown
- see Goodrich & Tamassia

Selection sort (1)

- Idea: Find the least element in the array, swap it into the leftmost slot (where it belongs). Repeat this, now ignoring the leftmost slot.

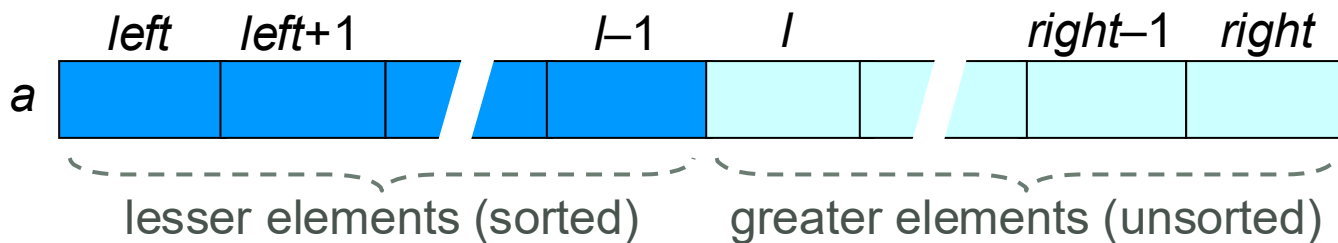
Selection sort (2)

- Selection sort algorithm:**

To sort $a[\text{left} \dots \text{right}]$ into ascending order:

- For $l = \text{left}, \dots, \text{right}-1$, repeat:
 - Set p such that $a[p]$ is the least of $a[l \dots \text{right}]$.
 - If $p \neq l$, swap $a[p]$ and $a[l]$.
- Terminate.

- Loop invariant:**

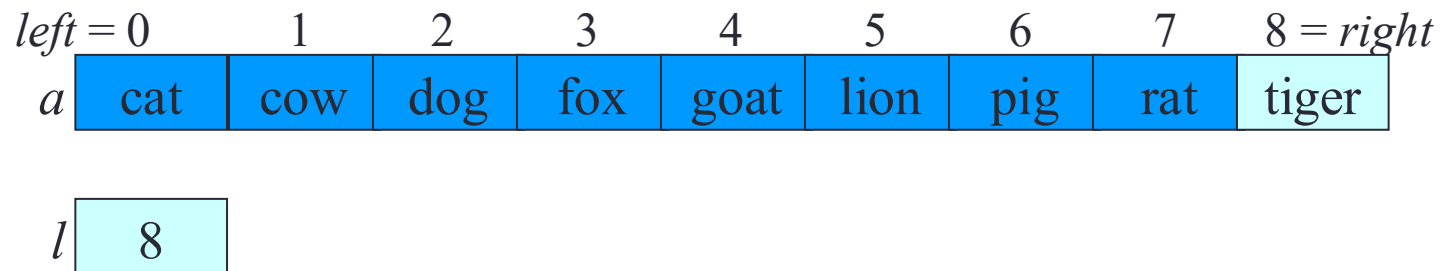


Selection sort (3)

- Animation:

To sort $a[\text{left} \dots \text{right}]$ into ascending order:

1. For $l = \text{left}, \dots, \text{right}-1$, repeat:
 - 1.1. Set p such that $a[p]$ is the least of $a[l \dots \text{right}]$.
 - 1.2. If $p \neq l$, swap $a[p]$ and $a[l]$.
2. **Terminate.**



Selection sort (4)

- Analysis (counting comparisons):

Let $n = \text{right} - \text{left} + 1$ be length of array to be sorted.

Step 1.1 performs $\text{right} - l$ comparisons.

This is repeated with $l = \text{left}, \dots, \text{right} - 2, \text{right} - 1$.

$$\begin{aligned}
 \text{No. of comparisons} &= (\text{right} - \text{left}) + \dots + 2 + 1 \\
 &= (n - 1) + \dots + 2 + 1 \\
 &= (n - 1)n/2 \\
 &= n^2/2 - n/2 \\
 &\approx n^2/2
 \end{aligned}$$

Time complexity is $O(n^2)$.

1. For $l = \text{left}, \dots, \text{right} - 1$, repeat:
 - 1.1. Set p such that $a[p]$ is the least of $a[l \dots \text{right}]$.
 - 1.2. If $p \neq l$, swap $a[p]$ and $a[l]$.
2. **Terminate.**

Selection sort (5): *implementation in Java*

```
public static <E extends Comparable<E>> void sort (E[] a,
                                                    int left, int right) {
    // Sort a[left-right] into ascending order
    for (int l = left; l < right; l++) {
        // nb l is the letter, 1 is the number
        int p = l;
        E least = a[p];
        for (int k = l+1; k <= right; k++){
            int comp = a[k].compareTo(least);
            if (comp < 0) {p = k; least = a[p];}
        }
        if (p != l) { a[p] = a[l]; a[l] = least; }
    }
}
```

克服

Aside: divide-and-conquer

- The **divide-and-conquer** strategy is effective for solving many problems.
- To solve a “hard” problem:
 - Break the problem down into two or more “easier” sub-problems.
 - Solve these sub-problems separately.
 - Combine their answers.
- The divide-and-conquer strategy naturally suggests recursive algorithms.

Merge-sort (1)

- Idea for sorting an array:
 - Divide the array into two subarrays of about equal length.
 - Sort the subarrays separately.
 - Merge the sorted subarrays.
- This is an application of the divide-and-conquer strategy.

Merge-sort (2)

- **Merge-sort algorithm:**

To sort $a[\textit{left} \dots \textit{right}]$ into ascending order:

1. If $\textit{left} < \textit{right}$:
 - 1.1. Let m be an integer about midway between $\textit{left}$ and $\textit{right}$.
 - 1.2. Sort $a[\textit{left} \dots m]$ into ascending order.
 - 1.3. Sort $a[m+1 \dots \textit{right}]$ into ascending order.
 - 1.4. Merge $a[\textit{left} \dots m]$ and $a[m+1 \dots \textit{right}]$ into auxiliary array b .
 - 1.5. Copy all elements of b into $a[\textit{left} \dots \textit{right}]$.
2. Terminate.

Merge-sort (3)

- Animation (treating 1.2 and 1.3 as single steps):

To sort $a[\text{left} \dots \text{right}]$ into ascending order:

1. If $\text{left} < \text{right}$:
 - 1.1. Let m be an integer about midway between left and right .
 - 1.2. Sort $a[\text{left} \dots m]$ into ascending order.
 - 1.3. Sort $a[m+1 \dots \text{right}]$ into ascending order.
 - 1.4. Merge $a[\text{left} \dots m]$ and $a[m+1 \dots \text{right}]$ into auxiliary array b .
 - 1.5. Copy all elements of b into $a[\text{left} \dots \text{right}]$.
2. **Terminate.**

$left = 0$	1	2	3	4	5	6	7	$8 = right$	
a	cat	cow	dog	fox	goat	lion	pig	rat	tiger

Merge-sort (4)

- For implementation see Moodle.

- Analysis (counting comparisons):

Let $n = \text{right} - \text{left} + 1$ be the length of the array to be sorted.

Let the total no. of comparisons required to sort a (sub)array of length m be $\text{comps}(m)$.

Step 1.2 takes about $\text{comps}(n/2)$ comparisons to sort the left subarray, since the subarray's length is about $n/2$.

Step 1.3 takes about $\text{comps}(n/2)$ comparisons to sort the right subarray, similarly.

Step 1.4 takes about $n-1$ comparisons to merge the subarrays.

Merge-sort (5)

- Analysis (*continued*):

Therefore:

$$\begin{aligned} \text{comps}(n) &\approx 2 \text{ comps}(n/2) + n - 1 && \text{if } n > 1 \\ \text{comps}(n) &= 0 && \text{if } n \leq 1 \end{aligned}$$

Solution:

$$\text{comps}(n) \approx n \log_2 n$$

Time complexity is $O(n \log n)$ (see next slide).

- **Space complexity** is $O(n)$, since step 1.4 needs an auxiliary array of length n .

- Omit next two slides if you don't want to see the proof – it is not examinable!!

Proof that merge-sort has complexity $O(n \log n)$

Use $c(n)$ to denote $\text{comps}(n)$ and assume that $n=2^m$ (and so $m=\log n$)

$$c(n) = 2c(n/2) + n - 1 \quad (1)$$

Apply equation (1) to $n/2$ to get:

$$c(n/2) = 2c(n/4) + n/2 - 1$$

Substitute this for $c(n/2)$ into equation (1) to get:

$$c(n) = 2(2(c(n/4) + n/2 - 1)) + n - 1$$

so

$$c(n) = 4c(n/4) + 2n - 3 \quad (2)$$

Similarly, applying equation (1) to $n/4$ we get

$$c(n/4) = 2c(n/8) + n/4 - 1$$

Substituting this into equation (2) we get:

$$c(n) = 8c(n/8) + 3n - 7 \quad (3)$$

(PTO)

Proof continued

The equations form a pattern, we can continue on with equation (4) etc. as follows:

$$c(n) = 2c(n/2) + n - 1 \quad (1)$$

$$c(n) = 4c(n/4) + 2n - 3 \quad (2)$$

$$c(n) = 8c(n/8) + 3n - 7 \quad (3)$$

$$c(n) = 16c(n/16) + 4n - 15 \quad (4)$$

$$c(n) = 32c(n/32) + 5n - 31 \quad (5)$$

▪

▪

$$c(n) = 2^m c(n/2^m) + mn - (2^m - 1) \quad (m)$$

Since $2^m = n$, $m = \log n$, we get

$$c(n) = nc(1) + n \log n - n + 1$$

And as $c(1) = 0$,

$$c(n) = n \log n - n + 1 = O(n \log n)$$

Quick-sort (1)

- Idea: Choose any element (called the **pivot**). Then **partition** the array into three subarrays such that:
 - the left subarray contains only elements less than (or equal to) the pivot;
 - the middle subarray contains only the pivot;
 - the right subarray contains only elements greater than (or equal to) the pivot.

Finally sort the left subarray and the right subarray separately.

- This is another application of the divide-and-conquer strategy.

Quick-sort (2)

- **Quick-sort algorithm:**

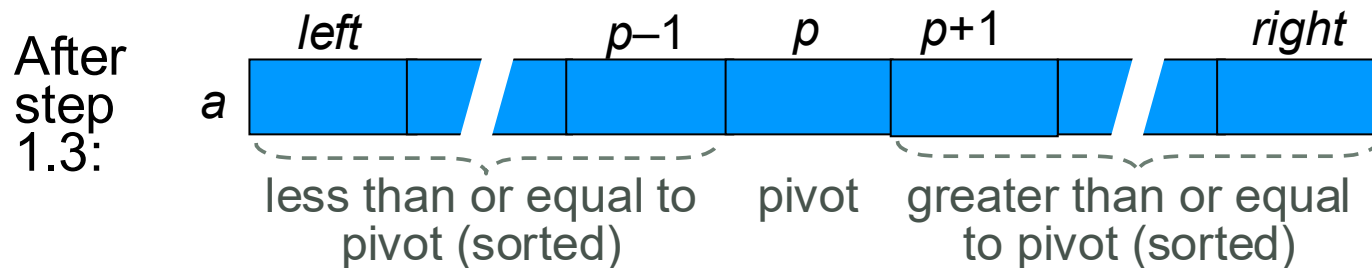
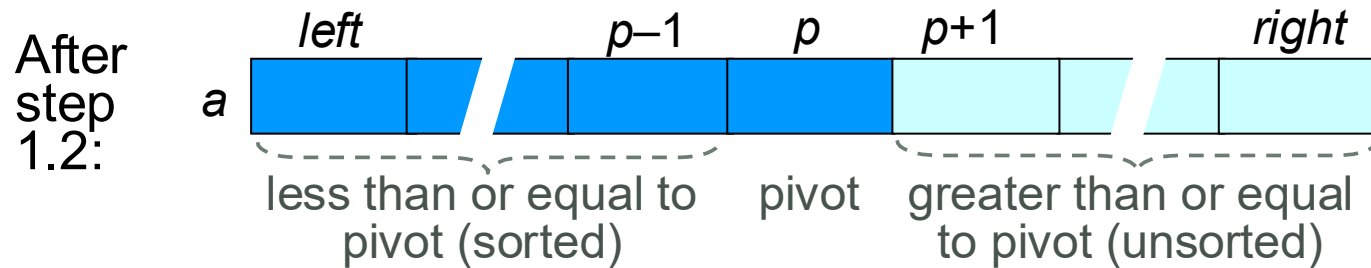
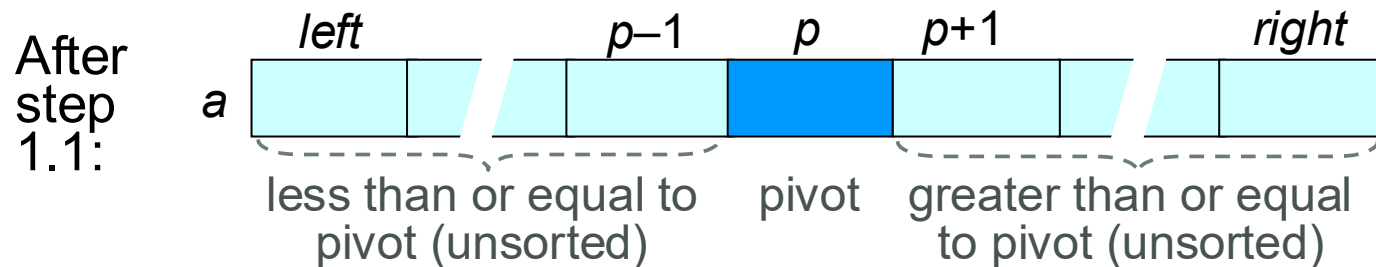
To sort $a[\textit{left} \dots \textit{right}]$ into ascending order:

1. If $\textit{left} < \textit{right}$:
 - 1.1. Partition $a[\textit{left} \dots \textit{right}]$ such that
 $a[\textit{left} \dots p-1]$ are all less than or equal to $a[p]$, and
 $a[p+1 \dots \textit{right}]$ are all greater than or equal to $a[p]$.
 - 1.2. Sort $a[\textit{left} \dots p-1]$ into ascending order.
 - 1.3. Sort $a[p+1 \dots \textit{right}]$ into ascending order.
2. Terminate.

$a[p]$ now
contains
the pivot

Quick-sort (3)

- Invariants:



Quick-sort (4)

- Animation:

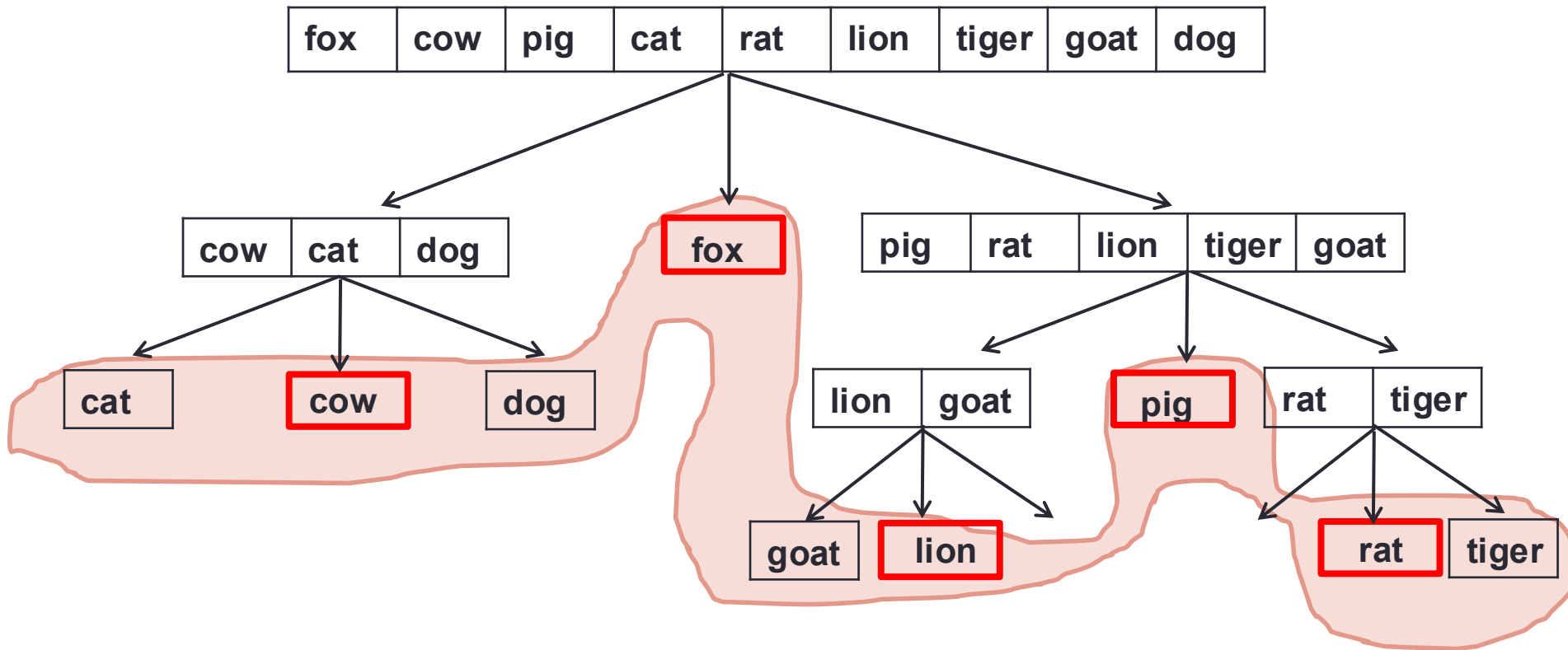
Assume this takes $a[\textit{left}]$ as the pivot, and doesn't reorder the elements put into the left or right subarray.

To sort $a[\textit{left} \dots \textit{right}]$ into ascending order:

1. If $\textit{left} < \textit{right}$:
 - 1.1. Partition $a[\textit{left} \dots \textit{right}]$ such that
 $a[\textit{left} \dots p-1]$ are all less than or equal to $a[p]$, and
 $a[p+1 \dots \textit{right}]$ are all greater than or equal to $a[p]$.
 - 1.2. Sort $a[\textit{left} \dots p-1]$ into ascending order.
 - 1.3. Sort $a[p+1 \dots \textit{right}]$ into ascending order.
2. **Terminate.**

$left = 0$	1	2	3	4	5	6	7	$8 = right$	
a	cat	cow	dog	fox	goat	lion	pig	rat	tiger

In full



Would in fact take place in position (i.e. inside the original array)

Quick-sort (5)

- Analysis (counting comparisons):

Let n be the length of the (sub)array to be sorted.

Let the total no. of comparisons required to sort a (sub)array of length m be $comps(m)$.

Assume that step 1.1 takes about $n-1$ comparisons to partition the array. (It's *not* a sort!)

Quick-sort (6)

- In the **best case**, the pivot always turns out to be the median element. So the left and right subarrays both have length about $n/2$. Steps 1.2 and 1.3 perform about $comps(n/2)$ comparisons each.

Therefore:

$$\begin{aligned}comps(n) &\approx 2\ comps(n/2) + n - 1 && \text{if } n > 1 \\comps(n) &= 0 && \text{if } n \leq 1\end{aligned}$$

Solution:

$$comps(n) \approx n \log_2 n$$

Best-case time complexity is $O(n \log n)$.

Quick-sort (7)

- In the **worst case**, the pivot always turns out to be the smallest element. So the right subarray has length $n-1$ whilst the left subarray is empty. Step 1.3 performs $\text{comps}(n-1)$ comparisons, but step 1.2 does nothing at all.

Therefore:

$$\begin{aligned}\text{comps}(n) &\approx \text{comps}(n-1) + n - 1 && \text{if } n > 1 \\ \text{comps}(n) &= 0 && \text{if } n \leq 1\end{aligned}$$

Solution:

$$\text{comps}(n) \approx (n^2 - n)/2 \approx n^2/2$$

Worst-case time complexity is $O(n^2)$.

- This case arises if the array is already sorted!
- Omit next two slides if you don't want to see the proof – it is not examinable!!

Proof that quick-sort has worst case complexity $O(n^2)$

For interest only.

Use $c(n)$ to denote comps(n)

$$c(n) = c(n-1) + n - 1 \quad (1)$$

Apply equation (1) to $n-1$ to get:

$$c(n-1) = c(n-2) + (n-1) - 1$$

so

$$c(n) = c(n-2) + n + (n-1) - 2 \quad (2)$$

Similarly, applying equation (1) to $n-2$ we get

$$c(n-2) = c(n-3) + (n-2) - 1$$

so

$$c(n) = c(n-3) + (n + (n-1) + (n-2) + (n-3)) - 2 \quad (3)$$

(PTO)

Proof that quick-sort has worst case complexity $O(n^2)$

contd.

Continuing in this way we get:

$$c(n) = c(n-1) + n - 1 \quad (1)$$

$$c(n) = c(n-2) + n + (n-1) - 2 \quad (2)$$

$$c(n) = c(n-3) + (n + (n-2) + (n-3)) - 3 \quad (3)$$

▪

▪

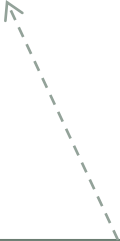
$$c(n) = c(n-(n-1)) + (n + (n-2) + \dots + (n-(n-2))) - (n-1) \quad (n-1)$$

i.e.

$$\begin{aligned} c(n) &= c(1) + (2+3+\dots+n) - n + 1 \\ &= c(1) + (1+2+\dots+(n-1)) \\ &= 0 + n(n-1)/2 \\ &= O(n^2) \end{aligned}$$

Quick-sort (8) – *Java implementation*

```
public static <E extends Comparable<E>> void quickSort(  
    E[] a, int left, int right)  
{  
    if(left<right){  
        int p=partition(a,left,right);  
        // assumes we have a partition algorithm that  
        // selects pivot, does the partitioning, and  
        // p is final position of pivot  
        quickSort(a,left,p-1);  
        quickSort(a,p+1,right);  
    }  
}
```



Will see how this is done in later slides.

See complete implementation on Moodle

Quick-sort partitioning (1)

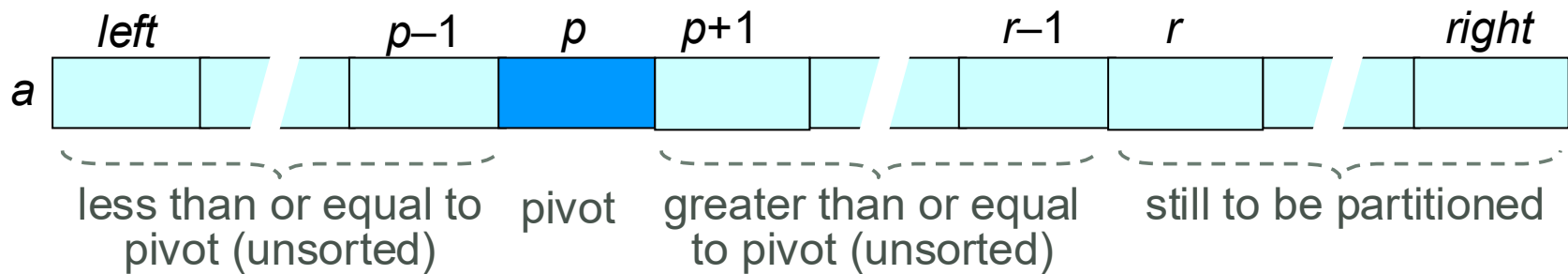
- **Quick-sort partitioning algorithm:**

To partition $a[\text{left} \dots \text{right}]$ such that $a[\text{left} \dots p-1]$ are all less than or equal to $a[p]$, and $a[p+1 \dots \text{right}]$ are all greater than or equal to $a[p]$:

1. Let $pivot$ be the element in $a[\text{left}]$, and set $p = \text{left}$.
2. For $r = \text{left}+1, \dots, \text{right}$, repeat:
 - 2.1. If $a[r]$ is less than $pivot$:
 - 2.1.1. Copy $a[r]$ into $a[p]$, $a[p+1]$ into $a[r]$,
and $pivot$ into $a[p+1]$.
 - 2.1.2. Increment p .
3. Terminate yielding p .

Quick-sort partitioning (2)

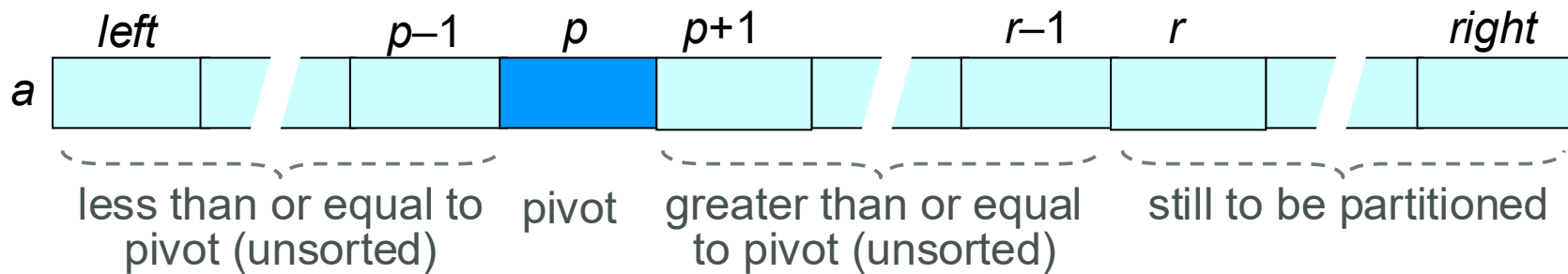
- Loop invariant:



- Note that other (and better) partitioning algorithms exist.
- Any algorithm that ends up with the final required condition (all elements $<$ pivot to left of pivot, all elements $>$ pivot to right of pivot) is legitimate.

Quick-sort partitioning Intuition (*Extra slide*)

- Loop invariant:



- **Quick-sort partitioning algorithm:**

To partition $a[left \dots right]$ such that $a[left \dots p-1]$ are all less than or equal to $a[p]$, and $a[p+1 \dots right]$ are all greater than or equal to $a[p]$:

1. Let *pivot* be the element in $a[left]$, and set $p = left$.
2. For $r = left+1, \dots, right$, repeat:
 - 2.1. If $a[r]$ is less than *pivot*:
 - 2.1.1. Copy $a[r]$ into $a[p]$, $a[p+1]$ into $a[r]$, and *pivot* into $a[p+1]$.
 - 2.1.2. Increment p .
3. Terminate yielding p .

Quick-sort partitioning: animation

To partition $a[\text{left} \dots \text{right}]$ such that $a[\text{left} \dots p-1]$ are all less than or equal to $a[p]$, and $a[p+1 \dots \text{right}]$ are all greater than or equal to $a[p]$:

1. Let *pivot* be the element in $a[\text{left}]$, and set $p = \text{left}$.
2. For $r = \text{left}+1, \dots, \text{right}$, repeat:
 - 2.1. If $a[r]$ is less than *pivot*:
 - 2.1.1. Copy $a[r]$ into $a[p]$, $a[p+1]$ into $a[r]$, and *pivot* into $a[p+1]$.
 - 2.1.2. Increment p .
3. **Terminate yielding p .**

cow	cat	dog	fox	rat	lion	tiger	goat	pig
-----	-----	-----	-----	-----	------	-------	------	-----

left = 0
right = 8

pivot = fox
p=3
r=8

Comparison of sorting algorithms

Algorithm	No. of comparisons	No. of copies	Time complexity	Space complexity
Selection sort	$\sim n^2/2$	$\sim 2n$	$O(n^2)$	$O(1)$
Insertion sort	$\sim n^2/4$	$\sim n^2/4$	$O(n^2)$	$O(1)$
Merge-sort	$\sim n \log_2 n$	$\sim 2n \log_2 n$	$O(n \log n)$	$O(n)$
Quick-sort (best) (worst)	$\sim n \log_2 n$ $\sim n^2/2$	$\sim 2n/3 \log_2 n$ 0	$O(n \log n)$ $O(n^2)$	$O(\log n)$ $O(n)$

Note that here space complexity is the ADDITIONAL/AUXILIARY space required to execute the sort. Insertion sort and Selection sort require no extra space, Merge-sort requires an array of size n (scratch array) which can be used at all steps of the recursion. Quick-sort requires a constant amount of space (to store pivot, and position of pivot) at every step in recursion tree. So the amount of space is proportional to the depth of the recursion tree, which depends on how well the array is already sorted, and the choice of the pivot. **Insertion sort: see extra slides.**